

Doc Site Portfolio Tutorial — Zensical Edition

Prepared for Edward McHam | Generated June 23, 2026

Table of Contents

Workflow Overview

Before you start

Step 1: Set up your environment

1.1 Choose your path

1.2 Create a project directory

1.3 Create a virtual environment

1.4A / 1.4B: Install Zensical

1.5A / 1.5B: Initialize your site

1.6A / 1.6B: Configure your site

1.7A / 1.7B: Preview your site

1.4C: Install Claude Code

1.5C: Authenticate

1.6C: Open your project directory

1.7C: Verify access

1.8: Customize your theme (optional)

1.9: Verify setup

Step 1 summary

Troubleshooting

Step 2: Plan your site

2.1 Define your portfolio goals

2.2 Choose documentation types

2.3 Select topics for your samples

2.4 Build your planning table

2.5 Understand your audience

2.6 Document your AI collaboration approach

2.7 Create docs/planning.md

Step 2 summary

Step 3: Generate your site structure

3.1 Plan your navigation structure

3.2 Configure navigation in zensical.toml

3.3 Create your directory structure

3.4 Draft your homepage

3.5 Create page templates

3.6 Draft your about page

3.7 Create your contact page

3.8 Test your structure

Step 3 summary

Step 4: Write your content

4.1 Principle: structure first, then content

4.2 Sample type: Localization / how-to guide

4.3 Sample type: Getting-started meta-sample

4.4 Third sample type

4.5 Combine your drafting with Claude's review

4.6 Write case studies for each sample

Step 4 summary

Quality checklist

Step 5: Refine

5.1 Use Claude Code for targeted review, not rewriting

5.2 Technique: clarity review

5.3 Technique: accuracy review

5.4 Technique: proprietary and overclaiming check

5.5 Technique: callouts

5.6 Final professional polish pass

Refinement checklist

Step 5 summary

Step 6: Deploy

6.1 Prerequisites

6.2 Create your GitHub repository

6.3 Initialize Git and create .gitignore

6.4 Commit and push

6.5 Set up the GitHub Actions deployment workflow

6.6 Enable GitHub Pages

6.7 Push and verify deployment

6.8 Verify the live site

6.9 Update your portfolio going forward

6.10 Set up your custom domain: transfer emcham.io

Step 6 summary

What you built

A note on your old repos

Doc Site Portfolio Tutorial — Zensical Edition

Adapted from the [rebeja Doc Site Portfolio tutorial](#) as a step-by-step procedure, written fresh for [Zensical](#), a modern static site generator. OONI is removed as a sample topic per your current plan.

Source change note: The original tutorial's scripts (`generate_content.py` , `improve_content.py`) and `config.yaml` workflow are not verified to work with Zensical. This rewrite keeps the manual prompting paths (Option B) and the Claude Code CLI path (Option C / Path C) as the verified routes, since those don't depend on the original repo's Python scripts. If you want to test the scripts against Zensical anyway, treat that as exploratory, not a step in this procedure.

WORKFLOW OVERVIEW

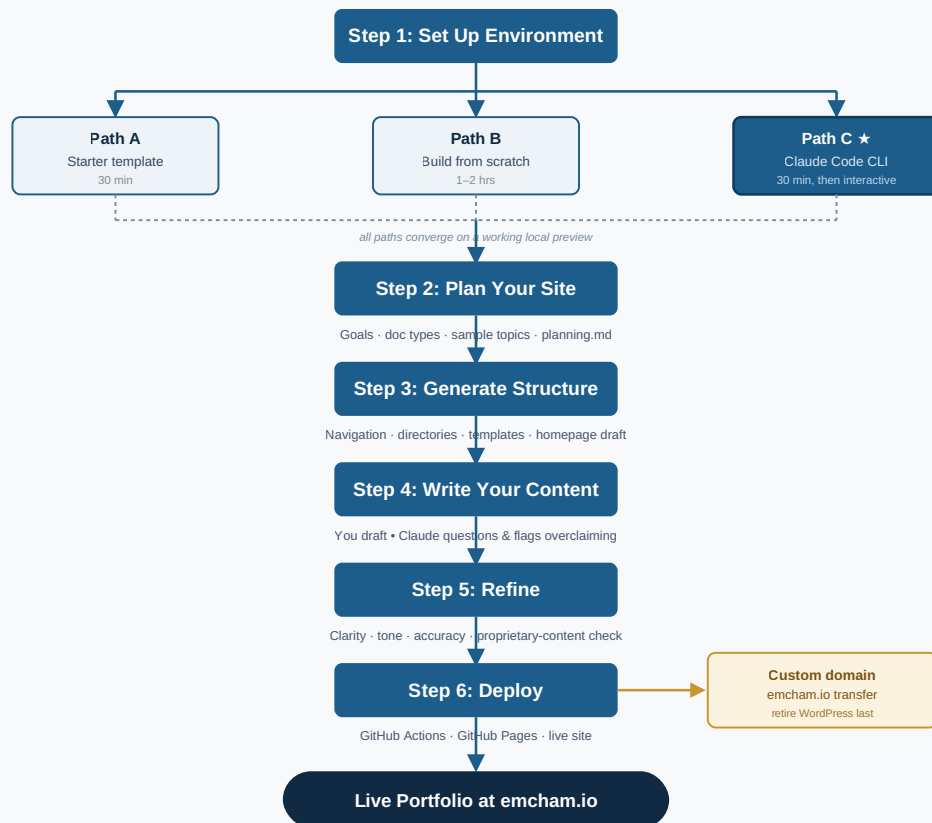


Figure 1. End-to-end build sequence, from environment setup through the live site at emcham.io. Path C (Claude Code CLI) is the route used in this procedure.

Before you start

Confirm you have:

- Python 3.10 or higher
- Git
- A GitHub account
- A code editor
- (Path C) A [Claude.ai](#) Pro or Max subscription and [Node.js](#) 18+

Verify Python and Git:

```
python3 --version  # 3.10 or higher
git --version      # any recent version
```

Step 1: Set up your environment

1.1 Choose your path

Path	Best for	Setup time
A: Starter template	Fastest start, basic structure pre-built	30 min
B: Build from scratch	Full control, learn every config option	1–2 hrs
C: Claude Code CLI	Conversational workflow, no scripts or API key	30 min, then interactive

Your memory shows you're using **Path C**. Steps 1.4C–1.7C below are the ones that apply to you; Paths A and B are included for completeness.

1.2 Create a project directory

```
mkdir my-portfolio
cd my-portfolio
```

1.3 Create a virtual environment

Create and activate a Python virtual environment:

```
python3 -m venv .venv
source .venv/bin/activate
```

On Windows:

```
python -m venv .venv
.venv\Scripts\activate
```

Confirm activation. The terminal prompt shows `(.venv)` when active.

1.4A / 1.4B: Install [Zensical](#)

Install Zensical with pip:

```
pip install --upgrade pip
pip install zensical
```

Zensical bundles its theme and search, so no separate theme package is required.

1.5A / 1.5B: Initialize your site

Scaffold a new project:

```
zensical new
```

This creates:

```
.
├── .github/workflows/
│   └── docs.yml
├── docs/
│   ├── index.md
│   └── markdown.md
└── zensical.toml
```

1.6A / 1.6B: Configure your site

Open `zensical.toml` and set the required and recommended fields:

```
site_name = "Your Name - Technical Writer"
site_description = "Software documentation portfolio showcasing API docs, tutorials, and
technical guides"
site_author = "Your Name"
site_url = "https://yourusername.github.io/my-portfolio/"

[theme]
features = [
  "navigation.tabs",
  "navigation.sections",
  "navigation.top",
  "search.highlight",
  "toc.follow",
]
```

`site_name` is the only required field. Add `site_url` next; several features, including social cards and sitemap generation, depend on it.

1.7A / 1.7B: Preview your site

```
zensical preview
```

Open `http://127.0.0.1:8000` in your browser. Confirm the placeholder content displays.

Press `Ctrl+C` to stop the preview server.

1.4C: Install Claude Code

```
npm install -g @anthropic-ai/claude-code
```

Confirm Node.js 18 or higher is installed:

```
node --version
```

1.5C: Authenticate

```
claude
```

Sign in through the browser prompt with your claude.ai account. No separate API key is required.

1.6C: Open your project directory

```
cd my-portfolio
claude
```

Claude Code starts in your project directory and reads your files directly.

1.7C: Verify access

In the Claude Code session, type:

```
What files are in this directory?
```

Confirm Claude lists your project files.

From here, you describe what you want in plain language instead of running scripts. Claude Code reads `zensical.toml`, your Markdown files, and your planning documents, and writes changes directly.

1.8: Customize your theme (optional)

Choose colors

```
[theme.palette]
scheme = "default"    # or "slate" for dark mode
primary = "indigo"    # blue, teal, purple, etc.
accent = "cyan"
```

Enable a light/dark toggle

```
[[theme.palette]]
media = "(prefers-color-scheme: light)"
scheme = "default"
primary = "indigo"
accent = "cyan"

[theme.palette.toggle]
icon = "material/brightness-7"
name = "Switch to dark mode"

[[theme.palette]]
media = "(prefers-color-scheme: dark)"
scheme = "slate"
primary = "indigo"
accent = "cyan"

[theme.palette.toggle]
icon = "material/brightness-4"
name = "Switch to light mode"
```

Verify the exact TOML table syntax for multiple palette entries against the current [Zensical color setup docs](#) before committing — Zensical's TOML schema is actively evolving as it leaves beta, and array-of-tables syntax for repeated palette blocks may change.

Choose fonts (optional)

```
[theme.font]
text = "Inter"
code = "Fira Code"
```

Test your customizations

```
zensical preview
```

Open `http://127.0.0.1:8000` and confirm the changes display.

1.9: Verify setup

```
which python      # should end in /.venv/bin/python
zensical --version # confirm Zensical is installed
zensical build     # should complete without errors
```

Step 1 summary

- ☐ Installed Python and Git
- ☐ Created and activated a virtual environment
- ☐ Installed Zensical (Path A/B) or Claude Code CLI (Path C)
- ☐ Scaffolded or opened your project directory
- ☐ Confirmed local preview works

Troubleshooting

Command not found: Activate your virtual environment: `source .venv/bin/activate` (`.venv\Scripts\activate` on Windows).

Port already in use: Run `zensical preview --port 8001`, or stop other processes using port 8000.

Theme not applying: Check `zensical.toml` for TOML syntax errors. Restart the preview server after any config change.

Step 2: Plan your site

Time estimate: 30–45 minutes.

2.1 Define your portfolio goals

Answer these questions in `docs/planning.md`:

- **Target job roles:** e.g., Senior Technical Writer, Documentation Engineer, API Documentation Specialist
- **Target industries:** e.g., developer tools, IT security/compliance, healthcare technology
- **Skills to demonstrate:** e.g., docs-as-code workflows, localization, information architecture, API documentation

2.2 Choose documentation types

Select 3–5 types across these categories. Include at least one per category you want to emphasize.

Category	Examples	Demonstrates
Reference	API reference, CLI reference, config reference	Precision, structured thinking
Task-oriented	Getting started guide, how-to guide, tutorial	User empathy, instructional design
Explanatory	Conceptual guide, comparison guide	Clarifying complex topics
Support	Troubleshooting guide, FAQ, error reference	Problem-solving

2.3 Select topics for your samples

Use one or a mix of these three approaches:

1. **Document open-source tools you use.** Authentic and verifiable, but check whether good documentation already exists before duplicating it.
2. **Create documentation for fictional products.** Full creative control, no conflicting official docs — but label it clearly as fictional.
3. **Document your own projects.** Genuine subject-matter expertise — but the project and code must be substantial enough to support real documentation.

2.4 Build your planning table

Documentation type	Topic	Approach	Audience	Purpose
[Type]	[Topic]	[Open source / Fictional / Own project]	[Audience]	[Purpose]

2.5 Understand your audience

Your portfolio serves two audiences:

- **Hiring managers and recruiters** — optimize for easy scanning, professional polish, range of documentation types.
- **Documentation team peers** — optimize for tooling knowledge (Zensical, Markdown, Git), docs-as-code understanding, process transparency.

2.6 Document your AI collaboration approach

State how you used AI for each sample. Example:

```
## How I used AI for this sample

**Tool**: Claude (via Claude Code CLI)

**Process**:
1. Drafted the outline myself
2. Asked Claude clarifying questions about scope and audience
3. Wrote the first draft myself
4. Asked Claude to flag overclaiming or unclear sections
5. Revised based on feedback

**AI contribution**: Clarifying questions and review feedback only – no AI-generated prose
```

Your stated approach is that you write your own drafts and use Claude for clarifying questions and flagging overclaiming or proprietary drift — not for generating prose. State that explicitly in each case study rather than the original tutorial's "AI contribution: 40% generation" framing, which assumes AI drafts the content.

2.7 Create `docs/planning.md`

```
# Portfolio Planning

## Goals

**Target roles**: [List 2-3 job titles]
**Target industries**: [List 2-3 industries]
**Key skills to demonstrate**: [List 3-5 skills]

## Documentation samples

| Type | Topic | Approach | Audience | Purpose |
|-----|-----|-----|-----|-----|
| [Type] | [Topic] | [Open source/Fictional/Own project] | [Audience] | [Purpose] |

## AI collaboration approach

**Primary AI tool**: Claude (Claude Code CLI)

**How I use it**:

- Clarifying questions before drafting
- Flagging overclaiming or proprietary content
- Reviewing structure and clarity

**What I do manually**:

- All drafting
- Technical accuracy verification
- Information architecture
- Final editing

## Timeline

**Week 1**: Setup and planning
**Week 2**: Create documentation samples
**Week 3**: Refine content and deploy
```

Step 2 summary

- ☐ Defined portfolio goals
- ☐ Selected 3–5 documentation types
- ☐ Chose topics and approaches
- ☐ Planned your AI collaboration approach
- ☐ Created `docs/planning.md`

Step 3: Generate your site structure

Time estimate: 45–60 minutes.

3.1 Plan your navigation structure

Recommended structure:

```
Home (index.md)
├─ About (about.md)
├─ Projects (projects/)
│   ├─ Overview (index.md)
│   ├─ Project 1
│   ├─ Project 2
│   └─ Project 3
├─ Samples (samples/)
│   ├─ Overview (index.md)
│   ├─ API Reference
│   ├─ Getting Started
│   ├─ CLI Reference
│   └─ Troubleshooting
└─ Contact (contact.md)
```

3.2 Configure navigation in `zensical.toml`

```
[[nav]]
title = "Home"
path = "index.md"

[[nav]]
title = "About"
path = "about.md"

[[nav.children]]
title = "Projects"

    [[nav.children.children]]
    title = "Overview"
    path = "projects/index.md"

    [[nav.children.children]]
    title = "Localization Sample"
    path = "projects/localization-sample.md"

[[nav]]
title = "Contact"
path = "contact.md"
```

Verify the exact nested-navigation TOML syntax against the [Zensical navigation reference](#) — confirm whether Zensical expects `[[nav]]` array-of-tables nesting as shown, or a different structure, before committing this section. If you omit `nav` entirely, Zensical scans `docs/` and builds navigation automatically; for a portfolio site, explicit navigation is worth the extra configuration because it demonstrates information-architecture skill to reviewers.

3.3 Create your directory structure

```
mkdir -p docs/projects
mkdir -p docs/samples

touch docs/projects/index.md
touch docs/projects/localization-sample.md
touch docs/projects/getting-started-meta-sample.md

touch docs/samples/index.md
touch docs/samples/getting-started.md
touch docs/samples/troubleshooting.md

touch docs/contact.md
```

Adjust filenames to match your planning table from Step 2.

3.4 Draft your homepage

Since you write your own drafts, use Claude Code to ask clarifying questions rather than generate content:

```
I'm drafting docs/index.md for my technical writing portfolio. Before I write it,
ask me clarifying questions about my target roles, the skills I want to highlight,
and the tone I should use. Don't draft the content — I'll write it after you ask
your questions.
```

Write `docs/index.md` yourself. Then, optionally, ask Claude to review it:

```
Review docs/index.md. Flag any claims that might overstate my experience level,
and flag anything that looks like it could expose proprietary employer information.
```

3.5 Create page templates

Create `templates/project-template.md` and `templates/sample-template.md` to keep your structure consistent across samples. Use the same section headers in every project case study so reviewers can scan quickly:

```
# [Project Name]

[One-sentence description]

## Project context

**Type**: [Documentation type]
**Audience**: [Target users]
**Approach**: [Open source / Fictional / Own project]
**Tools used**: [Zensical, Git, etc.]

## My approach

[How you researched and structured the content]

## AI collaboration

**How I used Claude**: [Clarifying questions / structure review / overclaiming checks]
**What I wrote myself**: [Everything – state this explicitly]

## View the documentation

[Link to the sample](../samples/[filename].md)

## What I learned

[Reflection]
```

3.6 Draft your about page

Use the same Socratic approach as the homepage: ask Claude to question your draft for clarity and honesty, not to generate it.

```
Review my About page draft for two things: (1) any phrasing that overstates my
expertise in areas I'm still building, and (2) any reference to proprietary
employer work that shouldn't be public. Don't rewrite my voice – flag issues
and explain why.
```

3.7 Create your contact page

```
# Contact

Get in touch to discuss documentation opportunities.

## Professional profiles

- **LinkedIn**: [your-linkedin-url]
- **GitHub**: [your-github-url]
- **Email**: [your-email]

## What I can help with

- [Your real specialties – e.g., API documentation, docs-as-code migration, localization support]

## Resume

[Download my resume (PDF)](files/resume.pdf)
```

3.8 Test your structure

zensical preview

Confirm:

- ☐ All navigation links resolve
- ☐ No broken page references
- ☐ Structure reflects your planning document
- ☐ Breadcrumbs display correctly

Step 3 summary

- ☐ Configured navigation in `zensical.toml`
- ☐ Created directory structure and placeholder files
- ☐ Drafted homepage, about, and contact pages yourself
- ☐ Used Claude for clarifying questions and review, not generation
- ☐ Created reusable templates
- ☐ Tested the structure locally

Step 4: Write your content

Time estimate: 2–3 hours (varies — you're writing this yourself, not generating it).

4.1 Principle: structure first, then content

1. Define structure and outline yourself.
2. Draft the content yourself.
3. Ask Claude clarifying questions where intent or audience is unclear, *before* or *while* drafting — not after, as a rewrite pass.
4. Ask Claude to flag overclaiming and proprietary drift.
5. Revise based on your own judgment.

This differs from the original tutorial's "AI generates, human edits" model. Your working approach inverts that: you generate, Claude questions and flags.

4.2 Sample type: Localization / how-to guide

This is your primary differentiator — based on real RTL/Farsi Drupal Wildfire theme work, generalized to avoid exposing proprietary employer content.

Structure first:

```
# How to Configure RTL Language Support in [Generalized Platform]

## Prerequisites
[What the reader needs before starting]

## Background: why RTL needs special handling
[Brief conceptual framing]

## Steps
### 1. [First config step]
### 2. [Second config step]
### 3. [Testing step]

## Common pitfalls
[Drawn from real experience, generalized]

## Related guides
```

Before drafting, ask Claude:

```
I'm writing a how-to guide on RTL/Farsi language support based on real Drupal theme work I did, but I need to generalize it so it doesn't expose any employer-specific implementation details. Ask me questions to help me figure out where the line is between "general RTL/Drupal knowledge" and "proprietary to my employer's specific build."
```

4.3 Sample type: Getting-started meta-sample

Document the process of building this Zensical portfolio itself.

Structure first:

```
# Getting Started: Building This Portfolio with Zensical

## What you'll learn
## Prerequisites
## Step 1: Install Zensical
## Step 2: Plan your content
## Step 3: Structure your site
## Step 4: Deploy to GitHub Pages
## What I'd do differently next time
```

This sample is naturally low-risk for proprietary drift since the subject is the tool itself, not employer work.

4.4 Third sample type

Your portfolio plan calls for a third sample beyond localization and the meta-sample (the OONI API sample has been removed per your current direction). Decide on a replacement topic — for example, the FrameMaker-to-Zensical migration article, or a CLI/config reference for a tool you use — and apply the same structure-first approach.

Open decision: Confirm with yourself what replaces the third sample slot before starting Step 4 in full. The migration article you've outlined (with the conditional-text problem and "when not to migrate" sections) is a strong candidate since it's already scoped.

4.5 Combine your drafting with Claude's review

What you provide:

- The outline and structure
- The first draft, in your own words
- Domain judgment on accuracy
- Final editing decisions

What Claude provides, when asked:

- Clarifying questions about audience, scope, and intent
- Flags for overclaiming relative to your actual experience level
- Flags for content that might be proprietary-adjacent
- Structural or clarity feedback, when you explicitly request it

4.6 Write case studies for each sample

```
# Case Study: [Sample Name]

## Overview

**Type**: [Documentation type]
**Topic**: [What you documented]
**Approach**: [Open source / Fictional / Own project / Generalized from real work]
**Audience**: [Who this is for]

## My process

### Research and planning
[How you gathered information; what you decided to generalize or exclude]

### Drafting
[Confirm: written by you]

### AI collaboration
**Tool**: Claude (Claude Code CLI)
**What I asked for**: [Clarifying questions / overclaiming checks / proprietary-drift checks]
**What changed as a result**: [Specifics]

## Skills demonstrated

- [Skill 1]
- [Skill 2]

## What I'd improve

- [Honest reflection]

[View the documentation →](../samples/[filename].md)
```

Step 4 summary

- ☐ Drafted 3 documentation samples in your own words
- ☐ Used Claude for clarifying questions and review, documented honestly
- ☐ Verified no proprietary employer content appears, including generalized derivatives
- ☐ Wrote a case study for each sample
- ☐ Confirmed positioning is honest about trajectory, not inflated expertise

Quality checklist

- ☐ Technical accuracy verified by you
- ☐ No proprietary or employer-specific content, including AI-paraphrased derivatives
- ☐ Consistent formatting across samples
- ☐ Tone matches your actual confidence level
- ☐ Case study written for each sample

Step 5: Refine

Time estimate: 1–2 hours.

5.1 Use Claude Code for targeted review, not rewriting

In your terminal Claude Code session:

```
Review docs/about.md for clarity and conciseness. Don't change technical terms,
don't change my voice, and don't add claims I haven't made. Flag anything
you're unsure about instead of changing it.
```

To check tone consistency across samples:

```
Compare docs/samples/localization-howto.md and docs/about.md. Flag any
inconsistency in voice or formality. Suggest fixes, but don't apply them –
I'll decide what to change.
```

Use `git diff` after any edit Claude makes directly, and review every change before accepting it.

5.2 Technique: clarity review

Ask Claude to identify, not fix:

- Overly long or complex sentences
- Unexplained jargon
- Passive voice where active would read better
- Redundant phrasing

Apply changes selectively. Evaluate every suggestion against: Is it actually clearer? Does it still sound like me?

5.3 Technique: accuracy review

```
Review this section for technical accuracy issues. Phrase concerns as questions
("Is it accurate that...?") rather than making the correction yourself.
```

Never accept a technical correction without verifying it yourself first.

5.4 Technique: proprietary and overclaiming check

This is your highest-priority refinement pass, given your stated requirements:

```
Read through all of docs/. Flag two categories of risk:
1. Anything that could expose proprietary employer information, including
   content that's been generalized or paraphrased from real work
2. Anything that overstates my expertise level relative to what I've actually
   done – especially in localization, API documentation, and docs-as-code

For each flag, explain your reasoning. Don't fix anything – I'll decide.
```

5.5 Technique: callouts

Zensical supports the following admonition syntax:

```
!!! note
    Additional context that helps understanding.

!!! tip
    A helpful best practice.

!!! warning
    An important caveat.
```

Use these for genuine prerequisites, common mistakes, and security-relevant notes. Don't overuse them.

5.6 Final professional polish pass

```
Do a final pass on docs/ for spelling, grammar, broken internal links, and
missing code syntax highlighting. List what you find – don't edit directly.
```

Refinement checklist

- ☐ Clarity reviewed
- ☐ Tone consistent across samples
- ☐ Technical accuracy verified by you
- ☐ **Proprietary content and overclaiming check completed and addressed**
- ☐ Code examples tested
- ☐ Internal links verified
- ☐ Case studies updated to reflect refinement

Step 5 summary

- ☐ Ran clarity and tone review through Claude Code
- ☐ Completed the proprietary/overclaiming check
- ☐ Verified all AI suggestions before accepting any
- ☐ Polished for professional presentation

Step 6: Deploy

Time estimate: 30–45 minutes.

6.1 Prerequisites

```
zensical build
ls -R site/
```

Confirm the build completes without errors.

6.2 Create your GitHub repository

1. Go to github.com/new
2. Name it (e.g., `emcham-portfolio`)
3. Set visibility to **Public** — required for free [GitHub Pages](#)
4. Do not initialize with a README
5. Click **Create repository**

6.3 Initialize Git and create `.gitignore`

```
git init
```

```
cat > .gitignore << 'EOF'
# Python
.venv/
__pycache__/
*.py[cod]

# Zensical
site/

# Environment
.env

# IDEs
.vscode/
.idea/

# OS
.DS_Store
Thumbs.db
EOF
```

6.4 Commit and push

```
git add .
git commit -m "Initial commit: portfolio site"
git remote add origin https://github.com/edwardmcham/emcham-portfolio.git
git branch -M main
git push -u origin main
```

6.5 Set up the GitHub Actions deployment workflow

Deployment runs through a [GitHub Actions](#) workflow, per [Zensical's publishing guide](#). The scaffold from `zensical new` already includes one at `.github/workflows/docs.yml`; confirm it matches this pattern:

```
name: Documentation
on:
  push:
    branches:
      - main
permissions:
  contents: read
  pages: write
  id-token: write
jobs:
  deploy:
    environment:
      name: github-pages
      url: ${ steps.deployment.outputs.page_url }
    runs-on: ubuntu-latest
    steps:
      - uses: actions/configure-pages@v5
      - uses: actions/checkout@v5
      - uses: actions/setup-python@v5
        with:
          python-version: "3.x"
      - run: pip install zensical
      - run: zensical build --clean
      - uses: actions/upload-pages-artifact@v4
        with:
          path: site
      - uses: actions/deploy-pages@v4
        id: deployment
```

6.6 Enable GitHub Pages

1. Go to your repository → **Settings** → **Pages**
2. Under **Build and deployment**, set **Source** to **GitHub Actions**

Zensical's recommended setup deploys directly through the Actions pipeline, so you do not select a branch here.

6.7 Push and verify deployment

```
git add .
git commit -m "Add deployment workflow"
git push
```

1. Go to the **Actions** tab in your repository
2. Confirm the "Documentation" workflow runs and completes with a green checkmark
3. Visit <https://edwardmcham.github.io/emcham-portfolio/>

6.8 Verify the live site

- ☐ Homepage loads
- ☐ Navigation works
- ☐ All pages render
- ☐ Internal and external links work
- ☐ Code blocks show syntax highlighting
- ☐ Site is responsive on mobile

6.9 Update your portfolio going forward

```
# Edit files, then:
git add .
git commit -m "Add CLI documentation sample"
git push
```

Each push to **main** triggers the Actions workflow automatically — there is no separate deploy command to run.

6.10 Set up your custom domain: transfer emcham.io

Since you plan to retire the WordPress site and point **emcham.io** at this new portfolio, complete these four steps in order.

Step 1: Configure DNS at your domain registrar.

For the apex domain (**emcham.io**):

```
A      @      185.199.108.153
A      @      185.199.109.153
A      @      185.199.110.153
A      @      185.199.111.153
```


For `www.emcham.io` :

```
CNAME    www    edwardmcham.github.io
```

Step 2: Configure GitHub Pages. Go to Repository → **Settings** → **Pages** → **Custom domain** → enter `emcham.io` → **Save**. Wait for DNS verification (can take 24–48 hours), then enable **Enforce HTTPS**.

Step 3: Update `zensical.toml`.

```
site_url = "https://emcham.io/"
```

Step 4: Commit and push to trigger a rebuild.

Before you cut over: Don't decommission the WordPress site until the new domain resolves correctly and you've confirmed there's no content on `emcham.io` (such as the `wp-css-arabic` example referenced in your resume) that needs to be preserved or redirected first.

Step 6 summary

- ☐ Created and pushed to a GitHub repository
- ☐ Set up the GitHub Actions deployment workflow
- ☐ Enabled GitHub Pages with source set to GitHub Actions
- ☐ Verified the live site
- ☐ Configured `emcham.io` as a custom domain
- ☐ Confirmed no content loss before retiring WordPress

What you built

- A live portfolio site on Zensical, deployed via GitHub Actions
- Documentation samples that are honestly scoped to your actual experience
- Case studies documenting your real drafting process and Claude's role as a questioner and reviewer, not a generator
- A custom domain transfer plan for `emcham.io`

A note on your old repos

You mentioned uncertainty about whether to delete or keep your earlier GitHub repos that didn't take off. This tutorial doesn't resolve that for you, but it's worth deciding deliberately rather than defaulting either way — for example, by considering whether each repo demonstrates effort and learning a hiring manager would value, or whether it's likely to read as abandoned and unfinished. If you want, ask Claude to help you think through each repo individually before deciding.